SISTEMAS OPERATIVOS AVANZADOS

Semáforos, Gestión de Entrada/Salida y Arquitectura de Sistemas de Ficheros

Índice

1. Sincronización y Semáforos **3**

1.1. El Semáforo: Concepto y Definición . . . 3

1.2. Operaciones Fundamentales . . . 3

1.2.1. 1. `sem_wait(sem)` (WAIT / P / LOCK) . . . 3

1.2.2. 2. `sem_signal(sem)` (SIGNAL / V / UNLOCK / POST) . . . 3

1.3. Patrones de Uso de Semáforos . . . 3

2. Gestión de Entrada/Salida: Conceptos y Abstracciones **4**

2.1. Taxonomía de Dispositivos en Linux . . . 4

2.1.1. 1. Dispositivo Físico (Nivel Hardware) . . . 4

2.1.2. 2. Dispositivo Lógico (Nivel Sistema Operativo) . . . 4

2.1.3. 3. Dispositivo Virtual (Nivel Usuario/Proceso) . . . 4

2.2. Drivers y Device Descriptors . . . 5

3. Arquitectura Interna: Tablas y Flujos **5**

3.1. Las Tres Tablas del Sistema de E/S . . . 5

3.1.1. 1. Tabla de Canales (TC) - `task_struct->files` . . . 5

3.1.2. 2. Tabla de Ficheros Abiertos (TFA) - `file_struct` . . . 5

3.1.3. 3. Tabla de Inodos (T-Inodos) - `inode_struct` . . . 5

3.2. El Flujo de la Syscall `open()` . . . 5

3.3. Comportamiento de `fork`, `dup` y Múltiples `open` . . . 6

4. Gestores de E/S: Solucionando el Bloqueo **6**

4.1. Concepto de Gestor . . . 6

4.2. Estructuras del Gestor . . . 6

4.3. Sincronización del Gestor (Esquema de 3 Semáforos) . . . 7

4.4. Traza Detallada: Lectura Síncrona con Gestor . . . 7

4.5. E/S Asíncrona . . . 8

5. Sistemas de Ficheros: Almacenamiento y Arquitectura **8**

5.1. Conceptos Básicos: Bloque vs Sector . . . 8

5.2. Estrategias de Asignación de Espacio . . . 8

5.2.1. 1. Asignación Contigua . . . 8

5.2.2. 2. Asignación Encadenada (Linked) . . . 8

5.2.3. 3. FAT (File Allocation Table) . . . 9

5.2.4. 4. Asignación Indexada (I-nodos) 9
5.2.5. 5. Asignación Multinivel (Unix/Ext2/Ext3/Ext4) 9
5.3. Directorios y Enlaces 9
5.4. Arquitectura en Capas de Linux (VFS) 9
5.4.1. 1. Capa VFS (Virtual File System) 10
5.4.2. 2. Capa SFS (Specific File System) 10
5.4.3. 3. Buffer Cache / Page Cache (Vital para rendimiento) 10

6. Comunicación entre Procesos (IPC) 10
6.1. Sockets y Tuberías (Pipes) 10

1. Sincronización y Semáforos

Aunque los threads comparten memoria y recursos, su ejecución es asíncrona. Esto introduce **condiciones de carrera**: situaciones donde el resultado final depende del orden incontrolado de ejecución. Para evitar esto, necesitamos mecanismos de sincronización. En este parcial, la herramienta fundamental es el **Semáforo**.

1.1. El Semáforo: Concepto y Definición

Un semáforo no es más que una variable entera protegida por el sistema operativo que actúa como un contador de recursos disponibles, junto con una cola de procesos bloqueados.

Definición: Estructura de un Semáforo

- **Contador (int):** Indica cuántas unidades del recurso están disponibles.
- **Cola de Bloqueados (Queue):** Lista de PCBs (Process Control Blocks) de los procesos/threads que intentaron acceder al recurso y no pudieron.

El acceso a esta estructura es **atómico**. No se puede interrumpir una operación sobre un semáforo a la mitad.

1.2. Operaciones Fundamentales

Existen dos (y solo dos) operaciones principales para interactuar con un semáforo. Dependiendo de la literatura, pueden llamarse P/V, Wait/Signal o Down/Up.

1.2.1. 1. `sem_wait(sem)` (WAIT / P / LOCK)

Esta operación se usa para **solicitar acceso** a un recurso o esperar un evento.

1. Decrementa el contador del semáforo: `sem->count-`.
2. Verifica el valor del contador:
   - Si **contador** $\geq$ **0**: El recurso estaba libre. El proceso continúa su ejecución (ha .ªdquirido.el semáforo).
   - Si **contador** $<$**0**: El recurso no está disponible. El proceso que ha llamado a la función se **bloquea** (pasa a estado BLOCKED) y se añade a la cola del semáforo. El SO llama al planificador para ejecutar otro proceso.

1.2.2. 2. `sem_signal(sem)` (SIGNAL / V / UNLOCK / POST)

Esta operación se usa para **liberar un recurso** o avisar de que un evento ha ocurrido.

1. Incrementa el contador del semáforo: `sem->count++`.
2. Verifica el valor del contador:
   - Si **contador** $\leq$ **0**: Significa que había procesos esperando (el contador era negativo antes de incrementar). El SO selecciona un proceso de la cola de bloqueados y lo **despierta** (pasa a estado READY).
   - Si **contador** $>$**0**: No había nadie esperando. El semáforo simplemente queda con un recurso .exttra"disponible.

1.3. Patrones de Uso de Semáforos

Es vital distinguir *para qué* usamos el semáforo, lo cual depende de su valor inicial (N).

- **Exclusión Mutua (Mutex) [$N = 1$]:** Se usa para proteger una **Región Crítica** (código que accede a datos compartidos). Solo un proceso puede entrar a la vez.

```
    sem_wait(mutex); // Bloquea si alguien ya est  dentro
    // ... Regi n Cr tica (acceso a variables globales) ...
    sem_signal(mutex); // Libera para el siguiente
```

- **Sincronización [$N = 0$]:** Se usa para forzar un orden de ejecución. .[El] proceso A debe esperar a que B llegue a un punto". El que llega primero al `wait` se espera.

```
    // Proceso A (Consumidor)      // Proceso B (Productor)
    sem_wait(sinc);                // ... produce datos ...
    // Contin a solo tras signal    sem_signal(sinc);
```

- **Gestión de Recursos [$N > 1$]:** Controla el acceso a un pool de recursos finitos (ej. 5 impresoras). Permite que hasta N procesos entren simultáneamente.

2. Gestión de Entrada/Salida: Conceptos y Abstracciones

El objetivo supremo del subsistema de E/S es la **Independencia de Dispositivo**. Un programador debe poder escribir `write(fd, "hola", 4)` sin importarle si `fd` representa un archivo en un disco magnético, un SSD, una memoria USB, una terminal o un socket de red.

2.1. Taxonomía de Dispositivos en Linux

Para lograr esta abstracción, el SO divide la realidad en tres capas conceptuales:

2.1.1. 1. Dispositivo Físico (Nivel Hardware)

Es el componente tangible (el "hierro"). Ejemplos: Disco Duro SATA, Teclado USB, Tarjeta Ethernet.

- Son gestionados exclusivamente por el **Device Driver** (controlador).
- El SO los identifica mediante códigos de hardware (Vendor ID, Product ID).
- Son invisibles para el usuario normal.

2.1.2. 2. Dispositivo Lógico (Nivel Sistema Operativo)

Es la abstracción que el SO presenta en el sistema de ficheros, normalmente bajo el directorio `/dev`. Representan una interfaz de acceso al driver.

- **Identificación (Major/Minor):** En lugar de nombres, el kernel usa dos números:
  - **Major Number:** Identifica al **Driver** (el código) encargado de gestionarlo. Todos los discos IDE podrían tener Major 3.
  - **Minor Number:** Identifica la **Instancia** específica dentro de ese driver. El primer disco sería Minor 0, el segundo Minor 1.
- **Creación (`mknod`):** La syscall `mknod` crea un fichero especial en disco (un i-nodo) que no contiene datos, sino estos dos números (Major/Minor).

2.1.3. 3. Dispositivo Virtual (Nivel Usuario/Proceso)

Es la instancia de uso de un dispositivo lógico por parte de un proceso. Cuando un proceso abre un dispositivo lógico, obtiene un **Canal** o **File Descriptor (fd)**.

- Permite mantener el **estado dinámico** de la interacción. Por ejemplo, si dos procesos abren el mismo fichero, cada uno necesita su propio puntero de lectura (offset) para no interferirse. Ese offset vive en el dispositivo virtual.

2.2. Drivers y Device Descriptors

El núcleo del SO no sabe cómo hablar con un disco. Para ello usa una tabla de punteros.

- **Device Driver:** Es el código binario (módulo del kernel) proporcionado por el fabricante o la comunidad que sabe qué registros de CPU tocar para mover el cabezal del disco.
- **Device Descriptor (DD):** Es una estructura en memoria del kernel que actúa como çontrato". Contiene punteros a las funciones del driver (`op_open`, `op_read`, `op_write`).
- El **Major Number** actúa como índice en un vector global de Device Descriptors. `vector_dd[major].open()` llamará a la función correcta.

3. Arquitectura Interna: Tablas y Flujos

Para conectar un número entero (`fd`) en el programa de usuario con un sector físico en el disco, el SO atraviesa tres tablas fundamentales. Entender esto es la clave para resolver ejercicios de `fork`, `dup` y `open`.

3.1. Las Tres Tablas del Sistema de E/S

3.1.1. 1. Tabla de Canales (TC) - `task_struct->files`

- **Ámbito:** Privada por proceso (está en el PCB).
- **Estructura:** Un array simple de punteros. El índice del array es el `fd`.
- **Función:** Traducir el `fd` del usuario a una entrada en la Tabla de Ficheros Abiertos.

3.1.2. 2. Tabla de Ficheros Abiertos (TFA) - `file_struct`

- **Ámbito:** Global del Sistema Operativo (en memoria RAM).
- **Función:** Representa el **Dispositivo Virtual**. Guarda el estado de la sesión.
- **Campos Críticos:**
  - **Offset (Puntero L/E):** La posición actual en el fichero (ej. byte 1024). *Nota: Esto implica que si varios procesos comparten esta entrada, comparten el puntero de lectura.*
  - **Modo de Apertura:** (`O_RDONLY`, `O_WRONLY`...).
  - **Ref Count:** Cuántos canales apuntan a esta entrada. Se borra solo cuando llega a 0.
- Apunta a la Tabla de Inodos.

3.1.3. 3. Tabla de Inodos (T-Inodos) - `inode_struct`

- **Ámbito:** Global (Caché en memoria).
- **Función:** Representa el **Dispositivo Lógico**. Es una copia en RAM del i-nodo del disco para acceso rápido.
- **Contenido:** Metadatos estáticos (Tamaño, Permisos, Dueño) y el puntero al **Device Descriptor**.

3.2. El Flujo de la Syscall `open()`

Cuando ejecutamos `fd = open(/dev/midisco", O_RDONLY)`:

1. **Resolución de Ruta:** El SO navega por los directorios hasta encontrar el i-nodo de `midisco`.
2. **Carga de Inodo:** Si no está en memoria, carga el i-nodo en la **Tabla de Inodos**. Lee su Major/Minor y localiza el Device Descriptor correspondiente.
3. **Creación de Dispositivo Virtual:** Reserva una nueva entrada en la **TFA**.

- Inicializa **Offset = 0**.
- Establece modo `O_RDONLY`.
- Pone un puntero a la entrada de la T-Inodos.

4. **Asignación de Canal:** Busca la primera posición libre en la **TC** del proceso. Pone un puntero a la entrada de la TFA recién creada.
5. **Retorno:** Devuelve el índice de la TC como `fd`.

3.3. Comportamiento de `fork`, `dup` y Múltiples `open`

- `fork()`**:** El proceso hijo es una copia del padre. Se copia su TC. Los punteros de la TC del hijo apuntan a las **MISMAS entradas de la TFA** que el padre.
  - **Efecto:** Padre e hijo comparten el offset. Si el padre lee 10 bytes, el offset avanza 10 bytes para el hijo también.
- `dup(old_fd)`**:** Crea una nueva entrada en la TC del mismo proceso (un nuevo `fd`), pero que apunta a la **MISMA entrada de la TFA** que `old_fd`.
  - **Efecto:** Comparten offset.
- `open()` **dos veces sobre el mismo fichero:** Cada llamada a `open` crea una **NUEVA entrada en la TFA**.
  - **Efecto:** Tienes dos `fd` totalmente independientes. Cada uno tiene su propio offset en 0. Leer de uno no afecta al otro.

4. Gestores de E/S: Solucionando el Bloqueo

En un sistema básico, si un proceso llama a `read()`, el kernel llama al driver, y el driver espera al disco. Como el disco es millones de veces más lento que la CPU, el proceso retiene la CPU sin hacer nada útil (espera activa) o simplemente bloquea el flujo del sistema ineficientemente.

Para optimizar esto, introducimos el concepto de **Gestor de Dispositivo (I/O Manager)**.

4.1. Concepto de Gestor

Un Gestor es un **hilo del kernel** (Daemon) especializado en un dispositivo. Su trabajo es gestionar una cola de peticiones.

- Permite desacoplar la petición lógica (`read`) de la ejecución física.
- Permite implementar **E/S Asíncrona** y solapamiento CPU-E/S.
- Transforma la latencia de hardware en un **bloqueo pasivo** (semáforos) para el proceso de usuario, liberando la CPU para otros procesos.

4.2. Estructuras del Gestor

1. **IORB (Input/Output Request Block):** Es un "paquete"de datos que describe una petición. Contiene:
   - Tipo de operación (READ/WRITE).
   - Puntero al buffer del usuario.
   - Tamaño de datos.
   - **Un Semáforo Privado (`sem_operacion`):** Vital para bloquear al solicitante.
2. **Cola de Peticiones (`q_iorb`):** Lista enlazada de IORBs pendientes.
3. **Cola de Resultados (`q_iofin`):** Lista de IORBs completados con sus códigos de error/éxito.

4.3. Sincronización del Gestor (Esquema de 3 Semáforos)

Para que esto funcione sin condiciones de carrera y sin consumo de CPU inútil (busy waiting), se usan tres semáforos específicos. Es crucial memorizar sus valores iniciales y su función.

1. `mutex_colas` **(Valor Inicial: 1):** Semáforo tipo Mutex. Protege el acceso a `q_iorb` y `q_iofin`. Tanto el usuario (al encolar) como el gestor (al desencolar) deben adquirirlo.
2. `sem_gestor` **(Valor Inicial: 0):** Semáforo de Sincronización. Sirve para que el Gestor se **duerma** si no hay trabajo.
   - El Gestor hace `wait` al inicio de su bucle. Si la cola está vacía, se bloquea.
   - El Usuario hace `signal` después de poner un IORB, para "despertar.ªl gestor.
3. `sem_operacion` **(Valor Inicial: 0):** Semáforo de Sincronización (Privado por IORB). Sirve para que el **Usuario** se bloquee esperando SU resultado.
   - El Usuario hace `wait` tras enviar la petición. Se queda bloqueado.
   - El Gestor hace `signal` cuando el hardware ha terminado la tarea.

4.4. Traza Detallada: Lectura Síncrona con Gestor

Veamos paso a paso qué ocurre cuando un usuario llama a `read()`.

Flujo Crítico: Fase 1: El Usuario (Modo Kernel

1. El proceso llega al driver lógico. Crea una estructura **IORB** en memoria.
2. Inicializa el semáforo del IORB a 0.
3. Solicita acceso exclusivo a la cola: `sem_wait(mutex_colas)`.
4. Mete el IORB en `q_iorb`.
5. Libera la cola: `sem_signal(mutex_colas)`.
6. Avisa al gestor de que hay trabajo: `sem_signal(sem_gestor)`.
7. Se bloquea a esperar su dato: `sem_wait(iorb->sem_operacion)`.
8. *El proceso abandona la CPU. El planificador elige a otro (quizás al Gestor).*

Flujo Crítico: Fase 2: El Gestor (Kernel Thread)

1. Está en un bucle infinito. Espera trabajo: `sem_wait(sem_gestor)`.
2. Al despertar, pide acceso a la cola: `sem_wait(mutex_colas)`.
3. Saca el primer IORB de `q_iorb`.
4. Libera la cola: `sem_signal(mutex_colas)`.
5. Procesa el IORB: Llama al **Device Driver** físico.
6. El Driver programa el hardware (disco) y retorna inmediatamente (la E/S va por interrupciones).
7. El Gestor espera la interrupción (o se bloquea en otro semáforo interno del driver).
8. *... Tiempo físico de disco ...*
9. Llega la interrupción de hardware. El driver notifica al gestor.
10. El Gestor coloca el resultado en `q_iofin`.
11. Despierta al usuario dueño del IORB: `sem_signal(iorb->sem_operacion)`.

Flujo Crítico: Fase 3: El Usuario (Despierto)

1. El `sem_wait` retorna.
2. El proceso recoge el resultado de `q_iofin`.
3. Destruye el IORB.
4. La syscall `read` retorna los datos al espacio de usuario.

4.5. E/S Asíncrona

En la E/S Asíncrona, el usuario **no quiere bloquearse** en el paso 7 de la Fase 1.

- Existe una syscall `aio_read`: Encola el IORB y retorna inmediatamente un identificador de petición. El usuario sigue ejecutando código útil.
- Existe una syscall `aio_wait(id)` o `data_available(id)`: Es aquí donde el usuario pregunta "¿Ya está listo?". Si el gestor ya acabó, retorna los datos. Si no, es aquí donde se hace el `sem_wait` bloqueante.

5. Sistemas de Ficheros: Almacenamiento y Arquitectura

El Sistema de Ficheros (FS) es la capa que gestiona la persistencia. Transforma la visión del usuario (archivos y carpetas con nombres) en la visión del hardware (sectores físicos numéricos).

5.1. Conceptos Básicos: Bloque vs Sector

- **Sector:** Unidad mínima de transferencia física del disco (normalmente 512 Bytes). El disco no puede leer medio sector.
- **Bloque:** Unidad mínima lógica del Sistema Operativo. Es un múltiplo del sector (ej. 4KB = 8 sectores).
  - **Por qué bloques:** Reduce el tamaño de las estructuras de gestión (menos punteros que guardar) y mejora el rendimiento (lectura adelantada).
  - **Fragmentación Interna:** Si un fichero ocupa 1 byte, gasta un bloque entero (4KB). Se desperdician 4095 bytes. Es el precio a pagar por la eficiencia.

5.2. Estrategias de Asignación de Espacio

El problema fundamental es: dado un fichero que ocupa N bloques, ¿cuáles son esos bloques en el disco?

5.2.1. 1. Asignación Contigua

El fichero ocupa bloques físicamente adyacentes (ej. del 100 al 110).

- **Info necesaria:** Bloque inicio + Longitud.
- **Ventajas:** Lectura rapidísima (un solo movimiento de cabezal). Excelente para CD-ROMs.
- **Desventajas: Fragmentación Externa** severa (huecos libres donde no caben ficheros nuevos). Muy difícil ampliar un fichero (hay que moverlo entero si no hay hueco detrás).

5.2.2. 2. Asignación Encadenada (Linked)

Cada bloque contiene un puntero al siguiente bloque (como una lista enlazada).

- **Info necesaria:** Bloque inicio + Bloque final.
- **Ventajas:** No hay fragmentación externa. Fácil de ampliar.
- **Desventajas: Acceso Aleatorio pésimo**. Para leer el final del archivo, el SO debe leer todos los bloques anteriores uno a uno. Si se rompe un bloque, se pierde el resto del archivo.

5.2.3. 3. FAT (File Allocation Table)

Es una variante de la asignación encadenada. En lugar de guardar el puntero en el bloque de datos, se sacan todos los punteros y se guardan en una tabla maestra al inicio del disco (la FAT).

- **Ventajas:** La tabla se carga en RAM, así que seguir la cadena es rápido. Mejora el acceso aleatorio.
- **Desventajas:** La tabla crece con el tamaño del disco. Para discos modernos grandes, la FAT ocuparía gigabytes de RAM. Inviable hoy día.

5.2.4. 4. Asignación Indexada (I-nodos)

Cada fichero tiene un bloque especial llamado **Inodo** (Index Node). Este bloque contiene una lista de todos los bloques de datos del fichero.

- **Ventajas:** Acceso directo a cualquier bloque (cálculo de offset inmediato). Sin fragmentación externa.
- **Problema:** ¿Qué pasa si el fichero es gigante y la lista de bloques no cabe en un solo inodo?

5.2.5. 5. Asignación Multinivel (Unix/Ext2/Ext3/Ext4)

Es la solución estándar de Linux. El inodo usa un esquema híbrido para ser eficiente con ficheros pequeños y capaz con ficheros gigantes. Un inodo típico contiene 15 punteros:

- **12 Punteros Directos:** Apuntan directamente a bloques de datos. (Cubren ficheros hasta 48KB aprox). Acceso muy rápido.
- **1 Puntero Indirecto Simple:** Apunta a un bloque que está lleno de punteros a datos.
- **1 Puntero Indirecto Doble:** Apunta a un bloque lleno de punteros a bloques de punteros.
- **1 Puntero Indirecto Triple:** Tres niveles de indirección. Permite ficheros de Terabytes.

5.3. Directorios y Enlaces

Para el SO, un directorio no es más que un **fichero especial** cuyo contenido son datos binarios que el usuario interpreta como una lista.

- Cada entrada del directorio es un par: `(Nombre de Fichero, Número de Inodo)`.
- Esto separa el nombre del contenido real (metadatos).

Enlaces Duros (Hard Links) Como el directorio solo apunta al número de inodo, podemos tener dos entradas de directorio diferentes (ej. `/a/foto.jpg` y `/b/copia.jpg`) apuntando al **mismo número de inodo**.

- Ambos son el mismo fichero.
- El inodo tiene un contador (`link_count`).
- Al borrar un fichero (`unlink`), solo se borra la entrada del directorio y se decrementa el contador.
- El espacio en disco solo se libera cuando `link_count == 0` y ningún proceso lo tiene abierto.

5.4. Arquitectura en Capas de Linux (VFS)

Linux es capaz de leer un disco Ext4, un USB FAT32 y un CD ISO9660 simultáneamente. Esto se logra mediante una arquitectura modular.

5.4.1. 1. Capa VFS (Virtual File System)

Es la capa de abstracción. Define una interfaz genérica que todos los sistemas de ficheros deben cumplir.

- Trabaja con estructuras en memoria: **v-nodes** (inodos virtuales) y **d-entries** (entradas de directorio cacheadas).
- **Dentry Cache:** Guarda las rutas ya resueltas (ej. sabe que `/usr` es el inodo 50) para no leer el disco en cada acceso.
- **Gestión de Montaje:** Si el VFS detecta que una ruta cruza un **Punto de Montaje**, cambia automáticamente el puntero de operaciones para usar el driver del nuevo sistema de ficheros.

5.4.2. 2. Capa SFS (Specific File System)

Es el "driver del formato"(ej. `ext4.ko`, `ntfs.sys`).

- Sabe interpretar los bits del disco. Sabe qué es un inodo en Ext4 o qué es la FAT en FAT32.
- Traduce "Bloque Lógico del Fichero X"→ "Bloque Lógico del Disco Y".

5.4.3. 3. Buffer Cache / Page Cache (Vital para rendimiento)

El disco es lento. El SO reserva una gran parte de la RAM para actuar como caché de disco.

- **Funcionamiento:** Antes de leer del disco, el SFS pregunta a la Buffer Cache "¿Tienes el bloque X?".
- **Hit:** Retorna la memoria (nanosegundos).
- **Miss:** La Buffer Cache genera una petición real al driver físico, lee el bloque, lo guarda en RAM y se lo da al SFS.
- **Write Back:** Las escrituras se hacen en la caché (rápido) y se marcan como "sucias"(Dirty). Un hilo en segundo plano (`pdflush`) las escribe en disco más tarde.

6. Comunicación entre Procesos (IPC)

6.1. Sockets y Tuberías (Pipes)

Son mecanismos que el VFS presenta como ficheros pero que sirven para comunicar procesos.

- **Pipes:** Unidireccionales. Un proceso escribe en un extremo y otro lee del otro. Implementadas como un buffer circular en memoria del kernel.
- **Sockets:** Bidireccionales. Pueden comunicar procesos en la misma máquina (Unix Sockets) o en red (INET Sockets).
  - **Estructura `sk_buff`:** Es la estructura clave en Linux para gestionar paquetes de red. Pasa por las capas de Socket, Protocolo (TCP/IP) e Interfaz (Driver de red).